Valid Host, Subnet Address or Broadcast?

How to approach Section 4, Question 14 of the IP Addressing Practice

> **Q14.** For each, say **valid host**, **subnet address** or **broadcast**: (a) 192.168.4.63 /26 · (b) 192.168.4.64 /26 · (c) 172.16.8.0 /22 · (d) 172.16.9.0 /22 · (e) 172.16.11.255 /22 · (f) 10.0.255.255 /16 · (g) 10.0.255.255 /8

The idea: find the block the address sits in. If the address is the first address of the block, it's the subnet address. If it's the last, it's the broadcast. Anywhere in between, it's a valid host.

The 3-step method

1. **Turn /n into the cutting octet and the block size.** Work out which octet the cut falls in, and how many 1s land in it.
2. **Find the block range.** Use the 5 steps to get the **first** address (the subnet) and the **last** address (the broadcast).
3. **Compare.** Equals the first → **subnet address** · equals the last → **broadcast** · in between → **valid host** ✓

/n	Cut in octet	Ones in that octet	Mask octet	Block size
/26	4th (26 − 24 = 2)	2	192	64
/22	3rd (22 − 16 = 6)	6	252	4 (counted in the 3rd octet)
/16	On a boundary: the mask is 255.255.0.0, so the last 2 octets are all host bits			
/8	On a boundary: the mask is 255.0.0.0, so the last 3 octets are all host bits			

Shortcut for /8, /16 and /24: when the cut lands exactly on an octet boundary, no blocks are needed. Just look at the host octets: **all 0** → subnet address · **all 255** → broadcast · **anything else** → valid host.

Example 1: (a) 192.168.4.63 /26

1. /26 means 2 ones in the 4th octet, so the mask octet is 192 and the **block is 64**.
2. The blocks are 0–63, 64–127, 128–191 and 192–255. 63 is in **0–63**. First = 192.168.4.**0**, last = 192.168.4.**63**.
3. 192.168.4.63 is the **last** address, so it's the **broadcast**.

```
4th octet:  |  0 ... 63  | 64 ... 127 | 128 ... 191 | 192 ... 255 |
                    ^
                  .63 = last address of its block → BROADCAST
```

Binary check: the host bits are the last 6 bits of 63 = `00 111111`. They're **all 1s**, so it's the broadcast ✓.

Example 2: (c) 172.16.8.0 /22

1. /22 means 6 ones in the 3rd octet, so the mask octet is 252 and the **block is 4**, counted in the **3rd** octet.

2. The 3rd octet is 8. Counting in 4s: 0, 4, **8**, 12… so 8 is in the block **8–11**.
   First = 172.16.**8.0** (copy, start, then 0). Last = 172.16.**11.255** (copy, next start − 1 = 11, then 255).
3. 172.16.8.0 **equals the first** address, so it's the **subnet address**.

Draw the block as a line. Once you have it, (d) and (e) can be read straight off it:

```
172.16.8.0      ← subnet address (first)
172.16.8.1      ← first valid host
   ...
172.16.8.255    ← still inside the block
172.16.9.0      ← ?  (Q14d)
   ...
172.16.11.254   ← last valid host
172.16.11.255   ← broadcast (last)  (Q14e)
```

> **Key idea:** with /22 the host part is **10 bits**, which spill into the 3rd octet. So an address ending in .0 or .255 is **only** special if it is the very first or very last address of the whole block.

The trick in (f) and (g)

These are the **same address** with **different masks**. That's the point: **the mask decides the answer**.

Mask	Network part	Host part	Question to ask
/16	10.0	255.255	Are the host octets all 255?
/8	10	0.255.255	Are the host octets all 255?

Your turn

Try **(b), (d), (e), (f) and (g)** before you turn the page. Hint for (b): which block does 64 **start**?

Full worked solutions

Every part of Q14, step by step

Part	Address	Block	First (subnet)	Last (broadcast)	Answer
a	192.168.4.63 /26	64 in 4th	192.168.4.0	192.168.4.63	**Broadcast**
b	192.168.4.64 /26	64 in 4th	192.168.4.64	192.168.4.127	**Subnet address**
c	172.16.8.0 /22	4 in 3rd	172.16.8.0	172.16.11.255	**Subnet address**
d	172.16.9.0 /22	4 in 3rd	172.16.8.0	172.16.11.255	**Valid host ✓**
e	172.16.11.255 /22	4 in 3rd	172.16.8.0	172.16.11.255	**Broadcast**
f	10.0.255.255 /16	boundary	10.0.0.0	10.0.255.255	**Broadcast**
g	10.0.255.255 /8	boundary	10.0.0.0	10.255.255.255	**Valid host ✓**

(a) 192.168.4.63 /26 → broadcast

Block 64 in the 4th octet. 63 is in 0–63, and 63 is the **last** address of that block. Host bits `111111` = all 1s.

(b) 192.168.4.64 /26 → subnet address

Block 64: the blocks start at 0, **64**, 128, 192. 64 is the **start** of the block 64–127, so it's the **first** address. In binary, 64 = `01 000000`, so the host bits are `000000`, all 0s.

> (a) and (b) are neighbours, one apart, yet **neither** is a valid host. .63 ends one block and .64 starts the next.

(c) 172.16.8.0 /22 → subnet address

Block 4 in the 3rd octet. 8 ÷ 4 = 2 exactly, so 8 is the **start** of the block 8–11. Then the 4th octet is 0. So this is the **first** address of the block.

(d) 172.16.9.0 /22 → valid host ✓

8 ≤ 9 ≤ 11, so it's in the **same** block as (c): 172.16.8.0 to 172.16.11.255. It is not the first address (that's 8.0) and not the last (11.255), so it's in the middle. **Valid host**, even though it ends in .0.

3rd octet 9 = `0 0 0 0 1 0 0 1` 4th octet 0 = `0 0 0 0 0 0 0 0`

network bits host bits. The 10 host bits are `01 00000000`: there's a **1** in them, so they're **not** all 0s ✓.

(e) 172.16.11.255 /22 → broadcast

11 is in the block 8–11, and it's the **last** value (the next block starts at 12, and 12 − 1 = 11). With 255 after it, this is the **last** address of the block, the broadcast. The host bits are `11 11111111`, all 1s.

(f) 10.0.255.255 /16 → broadcast

/16 sits on a boundary, so the network is **10.0** and the host part is the last two octets: **255.255**. They are all 255 (all 1s), so this is the **broadcast** of network 10.0.0.0/16.

(g) 10.0.255.255 /8 → valid host ✓

/8 sits on a boundary, so the network is **10** and the host part is the last three octets: **0.255.255**. They are not all 255, because the first of them is 0. They're not all 0 either. So it's a **valid host** inside network 10.0.0.0/8, whose broadcast is 10.255.255.255.

Lesson from (f) and (g): the same 32 bits can be a broadcast under one mask and an ordinary host under another. Always find the block first.